GUIDED PROJECT ROADMAP

AquaFlow

Water Meter Reading & Billing System

A milestone-by-milestone learning journey in
Node.js microservices, Next.js and infrastructure engineering

Node.js + TypeScript | Next.js | PostgreSQL | RabbitMQ | Docker | Kubernetes | CI/CD | OpenTelemetry

13 milestones | ~29 weeks part-time | 7 services + 1 web app

Prepared for: learning developer & mentor

Version 1.0 | September 2026

Contents

1. Welcome & How to Use This Guide . . . 4

1.1 Who this is for . . . 4

1.2 Anatomy of a milestone . . . 4

1.3 Working agreements . . . 4

1.4 Weekly rhythm (suggested, 12-15 h/week) . . . 5

2. The Final Goal: What You Will Have Built . . . 6

2.1 Product vision . . . 6

2.2 Capabilities of the finished system . . . 6

2.3 Target architecture . . . 6

2.4 The life of a bill . . . 7

2.5 Recommended technology stack . . . 8

3. Domain Primer: How Water Billing Works . . . 9

3.1 Core concepts . . . 9

3.2 Roles and permissions . . . 9

3.3 Sample tariff (Residential, version 2026-01) . . . 10

3.4 Worked billing example . . . 10

3.5 Reading validation rules . . . 11

4. Roadmap at a Glance . . . 12

4.1 Suggested repository structure . . . 13

5. The Milestones . . . 14

Milestone 0 - Orientation & Domain Discovery . . . 14

Milestone 1 - Foundations: Monorepo, Tooling & the Customer Service . . . 16

Milestone 2 - Identity & Access: the Auth Service . . . 19

Milestone 3 - API Gateway & Containerising Everything . . . 21

Milestone 4 - Meter Reading Service . . . 23

Milestone 5 - Billing Engine . . . 25

Milestone 6 - Next.js Staff Portal . . . 27

Milestone 7 - Event-Driven Architecture: Broker, Payments & Notifications . . . 29

Milestone 8 - Customer Portal & Field Reader App . . . 31

Milestone 9 - Quality Gates & CI/CD . . . 33

Milestone 10 - Kubernetes & Infrastructure as Code . . . 35

Milestone 11 - Observability & Resilience . . . 37

Milestone 12 - Hardening, Performance & Capstone . . . 39

6. Capstone Acceptance Criteria . . . 41

6.1 Functional . . . 41

6.2 Architecture & code . . . 41

6.3 Infrastructure & operations . . . 41

6.4 Assessment rubric . . . 41

6.5 Mentor checkpoints . . . 42

Appendix A - Glossary of Technical Terms . . . 43

Appendix B - API Conventions . . . 44

Appendix C - Event Catalog (starter) . . . 45

Appendix D - ADR Template . . . 46

Appendix E - Learning Resources . . . 47

1. Welcome & How to Use This Guide

Welcome to **AquaFlow**, a water meter reading and billing platform for a fictional municipal water utility. Over roughly 29 weeks of part-time work you will build it from an empty folder to a system running on Kubernetes, made of independent Node.js services, a Next.js web application, a message broker and a full CI/CD and observability stack.

This is not a tutorial that you copy step by step. Each milestone tells you **what the finished result must do** and **how you will know it works**. It also tells you **which decisions are yours**. You will pick libraries, design data models, name things, shape the UI and write down why you chose what you chose. That is how professional engineers work, and it is the skill this project is meant to build.

1.1 Who this is for

- A developer who knows JavaScript fundamentals (async/await, modules, npm) and has built at least one small web app.
- Some SQL helps. No prior experience with microservices, Docker, Kubernetes or TypeScript is assumed. You will pick them up as you go.
- A mentor or senior reviewer is strongly recommended for the checkpoints in Section 6. The guide also works for self-study.

1.2 Anatomy of a milestone

Every milestone uses the same structure so you always know what is expected:

Section	What it gives you
Goal	A single sentence describing the outcome. If you remember nothing else, remember this.
Concepts	The engineering ideas the milestone is designed to teach. Research them before you start building.
What you are given	Fixed requirements, rules or reference material. Treat these as the client's requirements. They are not negotiable.
What you decide	Open design choices. Record the significant ones as Architecture Decision Records (ADRs, template in Appendix D).
Build tasks	An ordered list of the work. The order is a suggestion, and each task is deliberately high level.
Deliverables	The artifacts that must exist at the end (code, docs, diagrams, demos).
Definition of Done	Objective checks. You are not done until every item passes.
Stretch & reflection	Optional extras for when you are ahead, plus questions to answer in your dev journal.

1.3 Working agreements

- **Git from day one.** One repository (monorepo), small commits, *Conventional Commits* messages (`feat(billing): add tier calculator`), a branch per task and a pull request per task, even if you review it yourself.
- **Dev journal.** Keep `docs/journal.md`. Write 3 to 5 lines per working session: what you did, what confused you, what you learned. Answer the reflection questions here.

- **ADRs.** Any decision that would be expensive to reverse gets an ADR in `docs/adr/`. Expect 12 to 20 by the end.
- **Timebox research.** If you are stuck for more than 90 minutes, write down what you tried and ask for help. Knowing when to ask is a professional skill.
- **Demo every milestone.** Record a short (3 to 5 minute) screen capture, or give a live demo to your mentor. If you cannot demo it, it is not done.

1.4 Weekly rhythm (suggested, 12-15 h/week)

Day	Activity
Start of week	Re-read the milestone goal. Plan the week's tasks as GitHub Issues on a project board.
Mid-week	Build. Timebox concept research to about 20% of your hours.
End of week	Update the journal, tidy PRs, push, write a 5-line status note (done / next / blocked).
End of milestone	Demo, self-assess against the Definition of Done, meet your mentor at a checkpoint milestone.

2. The Final Goal: What You Will Have Built

Keep this section open throughout the project. It describes the finished system. Every milestone moves you closer to it.

2.1 Product vision

AquaFlow lets a water utility manage its customers and meters, collect monthly meter readings in the field (even without mobile signal), turn readings into accurate and auditable bills using tiered tariffs, collect payments and keep customers informed. Staff use a web portal, meter readers use a mobile-friendly offline app, and customers use a self-service portal.

2.2 Capabilities of the finished system

Area	Capabilities
Identity & access	Staff & customer login, JWT access/refresh tokens, 4 roles with a permission matrix, audit of sensitive actions.
Customers & meters	Customers, service accounts, addresses, zones/routes, meter inventory, installation, replacement, meter history.
Meter reading	Billing periods, route assignment, offline mobile reading capture with photos, validation rules, anomaly review queue, estimated reads.
Billing	Versioned tiered tariffs, bill runs per zone/period, fees & taxes, arrears, penalties, adjustments, PDF bills, per-account ledger.
Payments	Over-the-counter payments, online payments via a (mock or sandbox) gateway with webhooks, receipts, idempotent processing.
Notifications	Event-driven email/SMS for bill issued, payment received, overdue reminders and flagged readings.
Web app (Next.js)	Staff portal with dashboards, reader PWA (offline), customer portal (bills, usage chart, pay online).
Platform	Containerised services, Docker Compose locally, Kubernetes deployment, GitOps/CD, IaC, secrets management.
Operations	Structured logs, metrics, distributed traces, dashboards, SLOs & alerts, resilience patterns, backups.

2.3 Target architecture

The diagram below is the architecture you are working toward. You will not build it all at once. Services appear one by one and the infrastructure grows around them.

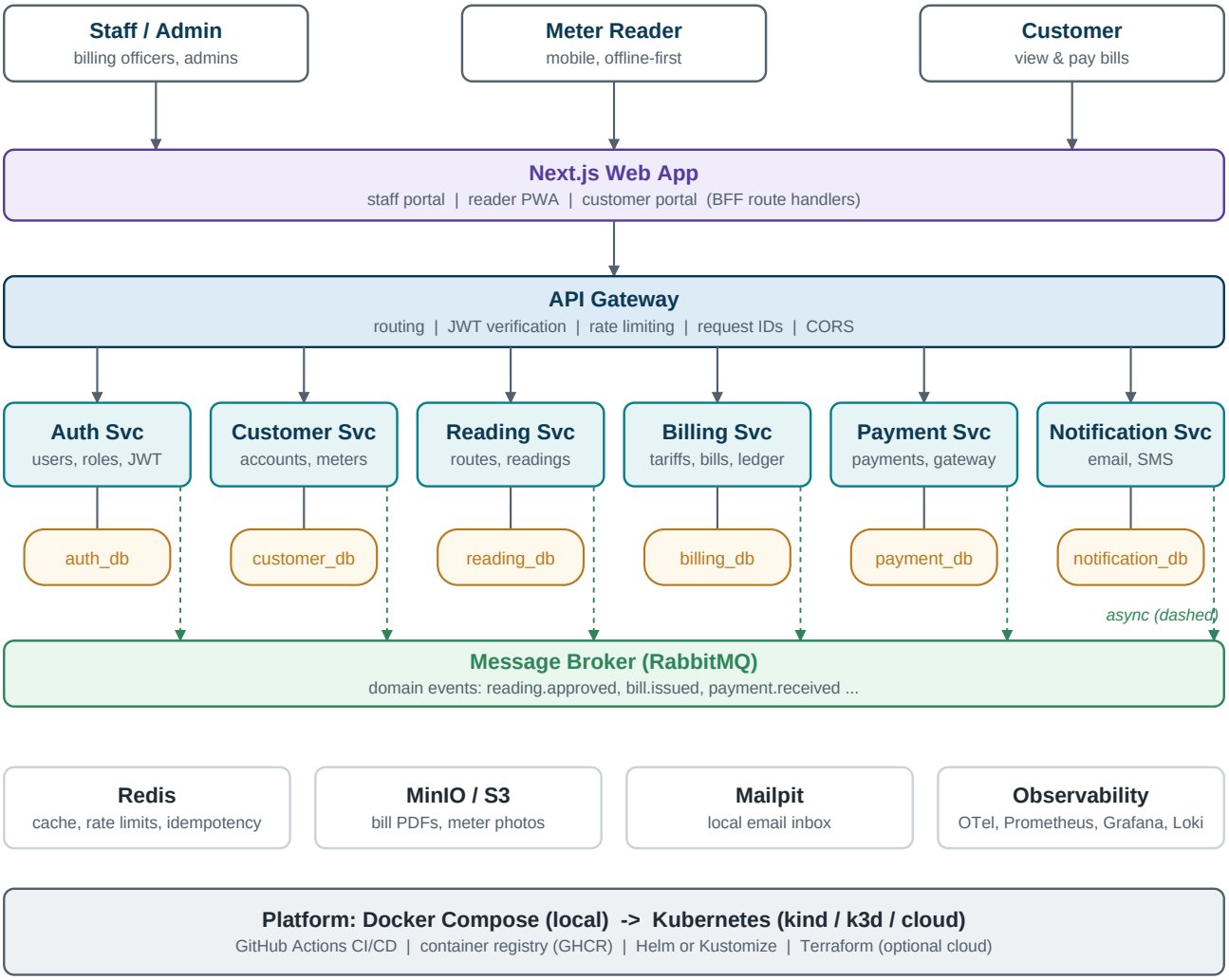
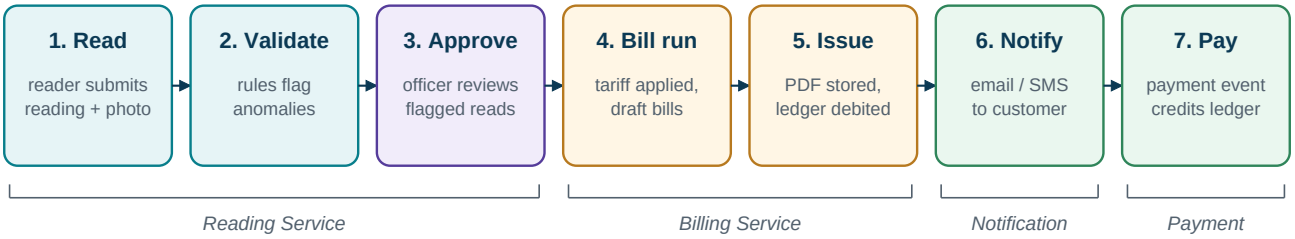


Figure 1. AquaFlow target architecture. Solid arrows are synchronous HTTP; dashed arrows are asynchronous events. Each service owns its database; no service reads another service's tables.

2.4 The life of a bill

This end-to-end flow is the heart of the system. Your final E2E test and your capstone demo both walk through it.



Everything from step 4 onward is triggered by events, not direct calls, once Milestone 7 is complete.

2.5 Recommended technology stack

Items marked **(choice)** are for you to decide. The recommendation is a safe default if you have no strong preference, but you still record the choice in an ADR.

Layer	Recommended	Alternatives / notes
Language & runtime	TypeScript on Node.js (current LTS)	Strict mode on. Plain JS is not recommended.
HTTP framework (choice)	Fastify	Express (most tutorials), NestJS (opinionated, heavier).
Data access (choice)	Prisma or Drizzle ORM	Knex / Kysely query builders; raw pg.
Database	PostgreSQL 16+	One logical database per service.
Validation	Zod	Share schemas between API and web where sensible.
Messaging (choice)	RabbitMQ	NATS JetStream, Redis Streams, Kafka (heavier ops).
Cache / rate-limit	Redis (or Valkey)	-
Object storage	MinIO locally, S3-compatible in cloud	-
Frontend	Next.js (App Router), React, Tailwind CSS	UI kit is your choice: shadcn/ui, Mantine, MUI...
Testing	Vitest/Jest, Supertest, Testcontainers, Playwright, Pact, k6	-
Containers	Docker, Docker Compose	Podman.
Orchestration (choice)	Kubernetes via kind or k3d; Helm	Kustomize; managed K8s (EKS/GKE/AKS/DOKS) for cloud.
CI/CD	GitHub Actions, GHCR, Argo CD (GitOps)	GitLab CI; Flux.
IaC (optional cloud)	Terraform / OpenTofu	Pulumi (TypeScript).
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Tempo/Jaeger	-

3. Domain Primer: How Water Billing Works

You cannot design good software for a business you do not understand. Read this section carefully in Milestone 0. It is the source of truth for the business rules you will implement.

3.1 Core concepts

Term	Meaning
Customer	The person or organisation responsible for paying. One customer may hold several accounts.
Service account	A billable connection at one service address (e.g., a house). Has a classification and a status.
Classification	Tariff category: *Residential*, *Commercial*, *Industrial*, *Government*. Determines the rates.
Meter	A physical device with a serial number, size and a register that only counts up. Installed on an account; may be replaced.
Zone / route	Accounts grouped geographically. A meter reader is assigned a route of about 100 to 300 meters per day.
Billing period	The window (usually monthly) for which consumption is billed. Each zone has a reading window, bill date and due date.
Reading	The meter register value on a date. **Consumption = current reading - previous reading** (in m^3).
Estimated reading	Used when the meter could not be read (no access, broken meter). Usually the average of recent consumption. Corrected when a real read happens.
Tariff	The price schedule: minimum charge, tiered (block) rates, fees and taxes. Tariffs change over time, so they are versioned.
Bill	An immutable statement of charges for one account and period. Corrections are made through adjustments, never by editing.
Ledger	The running list of debits (bills, penalties) and credits (payments, adjustments) for an account. Balance = sum of entries.
Arrears	Unpaid balance from previous bills, carried onto the next bill.
Penalty	Surcharge applied to overdue amounts after the due date (plus any grace period).
Rollover	When a meter register passes its maximum (e.g., 99999) and restarts at 0.

3.2 Roles and permissions

These roles are **given**. How you implement the checks is up to you (Milestone 2).

Capability	Admin	Billing officer	Meter reader	Customer
Manage staff users & roles	Yes	-	-	-
Create / edit customers, accounts, meters	Yes	Yes	-	-
View accounts & meters	Yes	Yes	Own route	Own only
Submit meter readings	Yes	-	Own route	Stretch: self-read
Review / approve flagged readings	Yes	Yes	-	-
Manage tariffs	Yes	-	-	-

Capability	Admin	Billing officer	Meter reader	Customer
Run billing, issue bills, adjustments	Yes	Yes	-	-
View bills & ledger	Yes	Yes	-	Own only
Record over-the-counter payments	Yes	Yes	-	-
Pay online	-	-	-	Own only
View audit log & dashboards	Yes	Dashboards	-	-

3.3 Sample tariff (Residential, version 2026-01)

Use this tariff for tests and seed data. Amounts are in a generic currency unit written as "$". Any currency works. Tiers are **cumulative blocks**: each cubic meter is charged at the rate of the block it falls into.

Component	Rule	Amount
Minimum charge	Covers the first 10 m^3 (charged even if consumption is lower)	$250.00 flat
Block 2	11 - 20 m^3	$30.00 / m^3
Block 3	21 - 30 m^3	$35.00 / m^3
Block 4	31 - 40 m^3	$42.00 / m^3
Block 5	41 m^3 and above	$50.00 / m^3
Environmental fee	10% of the basic charge (minimum + blocks)	percentage
Meter maintenance fee	Per bill, by meter size (1/2" meter)	$15.00 flat
VAT	12% of (basic charge + fees)	percentage
Late payment penalty	10% of the overdue amount, applied once, 3 days after the due date	percentage

Commercial and Industrial classifications use the same structure with higher rates. Define them yourself in Milestone 5 (for example, +40% and +80%).

3.4 Worked billing example

Account `RES-001-0042`, Residential. Previous reading **1,245**, current reading **1,282**, so consumption is **37 m^3**. The previous bill of $500.00 is unpaid and past due.

Line	Calculation	Amount
Minimum charge (first 10 m^3)	flat	250.00
Block 2 (11-20)	10 m^3 x 30.00	300.00
Block 3 (21-30)	10 m^3 x 35.00	350.00
Block 4 (31-37)	7 m^3 x 42.00	294.00
Basic charge		**1,194.00**
Environmental fee	10% x 1,194.00	119.40
Meter maintenance fee	flat	15.00
Subtotal		**1,328.40**
VAT	12% x 1,328.40 = 159.408, rounded half-up	159.41

Line	Calculation	Amount
Current charges		**1,487.81**
Arrears (previous unpaid bill)	carried forward	500.00
Late payment penalty	10% x 500.00	50.00
TOTAL AMOUNT DUE		**2,037.81**

> **Your acceptance test**
>
> Your billing engine must reproduce this bill **to the cent**. Put it in your test suite in Milestone 5 as a named test case. Also hand-calculate: (a) a 6 m^3 bill (minimum charge only), and (b) a 55 m^3 bill. Assume no arrears for both. Expected totals: (a) $324.80 and (b) $2,567.04. If your numbers differ, find out why before you write code.

3.5 Reading validation rules

These rules are **given** and are implemented in Milestone 4.

ID	Rule	System action
R1	Current reading is lower than the previous reading	Flag `NEGATIVE`, unless a rollover or meter replacement explains it
R2	Consumption is over 300% of the average of the last 3 periods	Flag `HIGH_USAGE` for review
R3	Zero consumption on an active account for 2 consecutive periods	Flag `ZERO_USAGE` (possible stuck meter)
R4	Reading date is outside the billing period's reading window	Reject with a clear error
R5	Duplicate submission for the same meter and period	Idempotent: return the existing reading, create nothing
R6	Reader records a no-access code (locked gate, dog, obstructed)	Create an `ESTIMATED` reading from the 3-period average
R7	Photo missing when the reading is flagged	Warn the reader in the app before submission

4. Roadmap at a Glance

The journey has five phases. Early milestones build one service deeply; middle milestones add services and connect them; later milestones are about running the system in a production-like way.

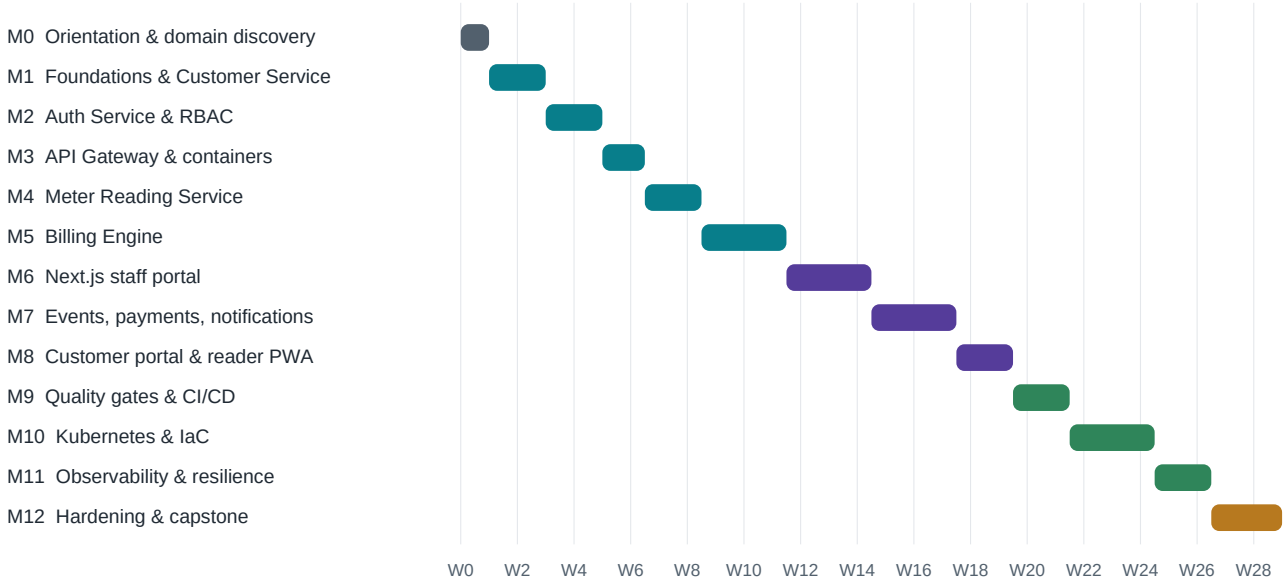


Figure 2. Timeline. **Discover Build services Integrate Operate Ship** - weeks assume 12-15 hours per week

#	Milestone	Weeks	Headline outcome	Main infra exposure
0	Orientation & domain discovery	1	Domain model, user stories, context map	-
1	Foundations & Customer Service	2	First tested service with its own DB	Docker Compose for Postgres
2	Auth Service & RBAC	2	JWT auth across services	Key management, JWKS
3	API Gateway & containers	1.5	Single entry point, all services containerised	Dockerfiles, networking, health checks
4	Meter Reading Service	2	Validated readings & consumption	Object storage (MinIO)
5	Billing Engine	3	Accurate, auditable bills & ledger	Background jobs
6	Next.js staff portal	3	Operational web portal	BFF pattern, secure cookies
7	Events, payments, notifications	3	Event-driven, reliable integration	RabbitMQ, outbox, DLQ
8	Customer portal & reader PWA	2	Self-service and offline field app	Service workers
9	Quality gates & CI/CD	2	Automated pipeline to a registry	GitHub Actions, GHCR, scanning
10	Kubernetes & IaC	3	System runs on Kubernetes from code	K8s, Helm, GitOps, Terraform
11	Observability & resilience	2	Traces, metrics, logs, SLOs	OTel, Prometheus, Grafana, Loki
12	Hardening & capstone	2.5	Secure, load-tested, documented, demoed	k6, backups, security scans

4.1 Suggested repository structure

This is a starting point. You may reshape it, but explain the change in an ADR.

```
aquaflow/
|-- apps/
|   `-- web/                      # Next.js: staff portal, reader PWA, customer portal
|-- services/
|   |-- gateway/
|   |-- auth/
|   |-- customer/
|   |-- reading/
|   |-- billing/
|   |-- payment/
|   `-- notification/
|       |-- src/ (routes/, domain/, repositories/, events/, config.ts, server.ts)
|       |-- test/
|       |-- migrations/
|       `-- Dockerfile
|-- packages/
|   |-- shared-config/            # tsconfig, eslint, prettier presets
|   |-- logger/                   # pino wrapper with request/trace ids
|   |-- auth-middleware/          # JWT verification + permission checks
|   `-- contracts/                # event schemas, shared DTO types (keep this SMALL)
|-- infra/
|   |-- compose/                  # docker-compose files, env templates
|   |-- k8s/                      # Helm charts or Kustomize bases/overlays
|   |-- terraform/                # optional cloud environment
|   `-- observability/            # Grafana dashboards, Prometheus rules
|-- docs/
|   |-- adr/  journal.md  glossary.md  architecture.md  runbooks/
`-- .github/workflows/
```

> **A warning about shared packages**
>
> Shared code creates coupling between services. Share **tooling and contracts** (logger, auth middleware, event schemas). Do **not** share **domain logic or database models**. If billing and reading both import the same `Meter` class, they are no longer independent services.

5. The Milestones

Each milestone starts on a new page. Work through them in order. Later milestones assume the earlier ones are finished.

Milestone 0 - Orientation & Domain Discovery

MILESTONE 0 | WEEK 1

Goal: Understand how a water utility reads meters and bills customers, and turn that understanding into a written domain model and service boundaries before writing any production code.

Most failed microservice projects fail at the whiteboard, not in the code. Services drawn around the wrong boundaries end up calling each other constantly and cannot change independently. In this week you only think, write and draw. It will save you weeks later.

Concepts you will learn

Domain-Driven Design basics	Bounded contexts	Ubiquitous language
User stories & acceptance criteria	Monolith vs microservices trade-offs	Architecture Decision Records

What you are given

- The domain primer (Section 3), including roles, tariff and validation rules.
- The target architecture (Figure 1) as a *reference*. Your boundaries may differ slightly if you can justify them.
- The ADR template (Appendix D).

What you decide (your responsibility)

- Details of your fictional utility: name, zones, billing cycle, account number format, grace periods.
- Exactly which data each service owns, and which data it only *references* by ID.
- Which assumptions you make where this guide is silent. Write every assumption down.

Build tasks

1. Read Section 3 twice. Write your own `docs/glossary.md` with at least 15 terms in your own words.
2. Write 12+ user stories across all four roles using *As a... I want... so that...*, each with 2-4 acceptance criteria.
3. Draw a context map: one box per service, the entities it owns, and the arrows (sync or async) between them. Any tool is fine: Excalidraw, draw.io, paper and a photo.
4. Hand-calculate the three bills in Section 3.4 in a spreadsheet. Make them match the expected totals.
5. Walk through two edge cases on paper: a meter replaced mid-period, and a customer who pays twice by mistake.
6. Write ADR-0001 *Adopt a microservice architecture*. Include honest **negative** consequences.
7. Create the empty Git repository, README with the project vision, and your journal.

Deliverables

- `docs/glossary.md`, `docs/user-stories.md`, context-map image, billing spreadsheet.
- ADR-0001 and a list of open questions and assumptions.

Definition of Done - you are finished when...

- You can explain, without notes, the full journey from a meter reading to a paid bill in under 2 minutes.
- Every entity in Section 3.1 has exactly one owning service on your context map.
- Your spreadsheet reproduces all three expected bill totals.
- Your mentor has reviewed the context map (Checkpoint A).

Common pitfalls

- Jumping straight into code.
- "Nano-services": a service per database table. Six services is plenty.
- Designing for millions of users. Design for clarity. Scaling comes later.

Stretch goals (optional)

- Event-storming session: write every domain event you can think of on sticky notes in past tense (*ReadingSubmitted*, *BillIssued*) and cluster them. Compare with Appendix C.

Reflection questions for your dev journal

? Why is a *meter* a separate entity from an *account*? What breaks if you merge them?

? Which service should own the "previous reading" used to compute consumption? Why?

? What are you giving up by choosing microservices over a modular monolith?

Milestone 1 - Foundations: Monorepo, Tooling & the Customer Service

MILESTONE 1 | WEEKS 2-3

Goal: A tested, documented Customer Service (customers, accounts, meters) written in TypeScript, backed by its own PostgreSQL database, and started locally with one command.

You will build one service *properly* before building six. The habits you set here (layered code, validation, migrations, tests, configuration from the environment) will be copied into every later service. Take your time.

Concepts you will learn

REST resource design	Layered architecture	Database migrations & seeding
Input validation	12-Factor configuration	Structured logging
Unit vs integration tests	Docker Compose basics	OpenAPI documentation

What you are given

- The endpoint list below and the API conventions in Appendix B (error format, pagination, naming).
- Business rule: an account may have **at most one active meter** at a time.
- Business rule: account status transitions are PENDING -> ACTIVE -> SUSPENDED/DISCONNECTED, and back to ACTIVE only from SUSPENDED.

What you decide (your responsibility)

- HTTP framework (Fastify / Express / NestJS). Write ADR-0002.
- ORM or query builder. Write ADR-0003.
- Monorepo tooling: npm workspaces, pnpm workspaces or Turborepo.
- Your folder structure inside the service, your naming conventions and your account-number format.

Build tasks

1. Initialise the monorepo: workspaces, strict TypeScript, ESLint, Prettier, shared tsconfig package, `.editorconfig`.
2. Create `infra/compose/docker-compose.yml` with PostgreSQL (plus Adminer or pgAdmin). Only the database runs in Docker for now.
3. Scaffold the Customer Service with config loaded and validated from environment variables at startup (fail fast on missing vars).
4. Design the schema and write migrations: customers, accounts, service_addresses, zones, meters, meter_installations (history!).
5. Implement the endpoints below in layers: route/controller -> service (business rules) -> repository (DB).
6. Add Zod validation and a global error handler that returns the Appendix B error format.
7. Add structured JSON logging (pino) with a request ID on every log line.
8. Write a seed script: 3 zones, 60 customers, 75 accounts, 80 meters (some in inventory, not installed).

9. Unit-test business rules; integration-test endpoints against a real Postgres (Testcontainers or a test compose file).
10. Expose OpenAPI docs at `/docs`.

Required endpoints (minimum)

Method & path	Purpose
POST /customers | GET /customers/:id | GET /customers?q=	Create, fetch, search customers
POST /accounts | GET /accounts?zone=&status=&page=	Open accounts; list with filters & pagination
PATCH /accounts/:id/status	Activate / suspend / disconnect (with reason)
POST /meters | GET /meters/:serial	Register meters in inventory
POST /accounts/:id/meter	Install a meter on an account (records initial reading)
POST /accounts/:id/meter/replace	Replace meter: final reading of old + initial reading of new
GET /healthz | GET /readyz	Liveness (process up) vs readiness (DB reachable)

Deliverables

- Customer Service source, migrations, seed script and tests.
- Root README section: *How to run locally* (should be at most 4 commands).
- ADR-0002 and ADR-0003.

Definition of Done - you are finished when...

- `docker compose up -d` followed by `npm run dev` gives a working API with seeded data.
- Installing a second meter on an account without replacing the first returns a **409 Conflict** with a helpful message.
- Meter replacement keeps full history; you can query which meter was on an account on any past date.
- Invalid input returns 400 with field-level errors; unknown IDs return 404; nothing returns a stack trace.
- Test coverage of the domain/service layer is at least 80%. All tests pass in a clean clone.
- Changing the DB password needs only an env var change, not a code change.

Common pitfalls

- SQL or ORM calls directly inside route handlers. Keep the layers.
- Using floating-point numbers for meter readings. Use integers or `NUMERIC`.
- Committing `.env` files. Commit a `.env.example` instead.

Stretch goals (optional)

- Generate the OpenAPI spec automatically from your Zod schemas.
- Audit columns (`created_by`, `updated_at`) and soft deletes.
- Pre-commit hooks with Husky + lint-staged.

Reflection questions for your dev journal

? If another service read your `customer_db` tables directly, what would break when you change the schema?

? What is the difference between liveness and readiness, and who uses each?

Milestone 2 - Identity & Access: the Auth Service

MILESTONE 2 | WEEKS 4-5

Goal: A dedicated Auth Service that issues signed JWTs, and a shared middleware that lets every other service verify tokens and enforce the role permission matrix without calling Auth on every request.

Authentication is the first cross-cutting concern that forces services to cooperate. You will learn why asymmetric signing (RS256/ES256) and a public JWKS endpoint let services trust tokens independently. This is stateless verification, and it is one of the main things that makes microservices work.

Concepts you will learn

Password hashing (argon2 / bcrypt)	Access vs refresh tokens	JWT, RS256 & JWKS
Role-based access control	Refresh-token rotation	Brute-force protection
Secrets handling	Shared internal packages	Security audit logging

What you are given

- The four roles and the permission matrix in Section 3.2.
- Passwords are never stored in plain text, never logged, and never returned by any API.
- Customers self-register only by proving they hold an account (account number + a code printed on a bill, or a seeded verification code).

What you decide (your responsibility)

- Token lifetimes and refresh strategy (rotation, reuse detection, storage).
- How a customer user is linked to their account(s): which service stores the link? Write an ADR.
- Whether permissions are embedded in the token or resolved from roles inside each service.
- Library choices (e.g., `jose`, `@fastify/jwt`, `argon2`).

Build tasks

1. Scaffold the Auth Service with its own `auth_db` (users, roles, refresh_tokens, login_events).
2. Implement login, refresh, logout, `GET /me` and admin-only staff user management.
3. Implement customer self-registration with account verification. Auth may need to ask the Customer Service to verify the account: your first service-to-service call.
4. Generate an RSA/EC key pair; sign access tokens; publish the public key at `/.well-known/jwks.json`.
5. Build `packages/auth-middleware`: fetch & cache JWKS, verify tokens, expose a `requirePermission('accounts:write')` guard.
6. Protect every Customer Service endpoint with the middleware, following the permission matrix. Readers see only their route's accounts; customers see only their own.
7. Add rate limiting / lockout on repeated failed logins and log security events.
8. Seed one user per role for local testing and document the credentials in the README (local only!).

Deliverables

- Auth Service, auth-middleware package, updated Customer Service.
- A permission-matrix test suite: for each role x endpoint, assert the expected 2xx/401/403.
- ADR on token strategy and ADR on customer-account linking.

Definition of Done - you are finished when...

- A *meter_reader* token gets **403** when creating an account; no token or an expired token gets **401**.
- A customer requesting another customer's account gets 403 or 404. It never gets the data.
- Rotating the signing key (with a `kid`) needs no redeploy of the Customer Service.
- A refresh token can be used only once; reusing an old one revokes the session.
- Grepping the logs for a test user's password finds nothing.

Common pitfalls

- Using HS256 with a secret shared across all services: any service could then forge tokens.
- Putting sensitive personal data in the JWT payload. Tokens are only encoded, not encrypted.
- Checking roles only in the frontend. The backend must always enforce them.

Stretch goals (optional)

- TOTP multi-factor authentication for admins.
- Password-reset flow (email is stubbed until Milestone 7).
- Try an external identity provider (Keycloak in Docker) on a branch and compare it with your own service.

Reflection questions for your dev journal

? Why is it risky if every service calls the Auth Service to validate every request?

? How would you instantly revoke access for a fired employee when tokens are stateless?

Milestone 3 - API Gateway & Containerising Everything

MILESTONE 3 | WEEKS 6-7 (1.5 WEEKS)

Goal: All client traffic enters through a single API gateway, and every service runs in its own container, so the whole system starts healthy with one `docker compose up`.

This is your first real infrastructure milestone. You will learn how containers package a service with its runtime, how they find each other on a network, and why a gateway keeps clients simple while services stay private.

Concepts you will learn

API gateway pattern	Reverse proxying	Multi-stage Dockerfiles
Container networking & DNS	Health checks & startup order	Timeouts
Correlation / request IDs	CORS	Rate limiting with Redis

What you are given

- Gateway responsibilities: route by path prefix, pre-verify JWTs, add/propagate `x-request-id`, rate limit, CORS, timeouts, uniform error responses.
- Services must not be reachable from the host except through the gateway (only the gateway publishes a port).
- Containers run as a non-root user.

What you decide (your responsibility)

- Build the gateway in Node (e.g., `@fastify/http-proxy`) or use Traefik / Nginx / Kong. Write an ADR comparing at least two options.
- Route and versioning scheme (e.g., `/api/v1/billing/...`).
- How you structure compose files (base + override for dev hot-reload?).

Build tasks

1. Write a multi-stage Dockerfile for each service (build stage with dev dependencies, slim runtime stage) plus a `.dockerignore`.
2. Extend docker-compose: Postgres (one DB per service via an init script), Redis, auth, customer, gateway.
3. Add compose health checks and `depends_on: condition: service_healthy`.
4. Run migrations automatically on startup or as a one-off compose job. Explain your choice.
5. Implement the gateway: routing, JWT pre-check, request IDs, CORS, Redis-backed rate limiting, upstream timeouts.
6. Propagate the request ID into every service's logs so one request can be followed across services.
7. Add a Makefile or root npm scripts: `up`, `down`, `logs`, `seed`, `test`, `reset`.

Deliverables

- Dockerfiles, compose files, gateway service, Makefile/scripts.
- `docs/architecture.md` started, with an updated diagram of what exists so far.

Definition of Done - you are finished when...

- From a fresh clone, `make up` (or equivalent) brings everything to *healthy* in under 2 minutes.
- Each service image is under 250 MB and runs as a non-root user.
- Stopping the Customer Service makes the gateway return a clean **503** within its timeout. It does not hang.
- Grepping all logs for one request ID shows its path through gateway and service(s).
- Only the gateway (and dev tools like Adminer) publish ports to the host.

Common pitfalls

- Using `localhost` for service-to-service URLs inside containers. Use the compose service name.
- Copying `node_modules` from the host into the image.
- Putting business logic in the gateway.

Stretch goals (optional)

- Hot reload inside containers using bind mounts and a dev override file.
- Replace your gateway with Traefik on a branch and compare configuration effort.

Reflection questions for your dev journal

? Which responsibilities belong in the gateway and which belong in each service? Where is the line?

? What is the difference between a container image and a running container?

Milestone 4 - Meter Reading Service

MILESTONE 4 | WEEKS 8-9

Goal: Meter readers can be assigned routes and submit readings (single or batched, with photos); the system validates them against the given rules, flags anomalies for review and produces final consumption per account per billing period.

This service has rich business rules and your first real data-ownership trade-off. It needs meter and account information owned by another service. Does it call that service every time, or keep its own copy? There is no free answer, and working it out is the point of this milestone.

Concepts you will learn

Data ownership across services	Synchronous service calls	Local read models / caching
Idempotency keys	Rule engines / strategy pattern	State machines
Pre-signed uploads to object storage	Batch APIs	Table-driven tests

What you are given

- The validation rules R1-R7 in Section 3.5.
- Reading states: SUBMITTED -> VALIDATED or FLAGGED; FLAGGED -> APPROVED or REJECTED (re-read requested); plus ESTIMATED.
- Consumption for a period with a meter replacement = (old final - old previous) + (new current - new initial).
- Photos are uploaded directly to object storage (MinIO) with pre-signed URLs, never streamed through the service.

What you decide (your responsibility)

- How the Reading Service gets meter/account/route data: a sync call per request, a cached copy, or a replicated read model. **Write the ADR carefully.** You will revisit it in Milestone 7.
- How rollover is detected (meter max digits stored where?).
- The shape of the batch-submission API used by the offline app later.
- How reviewers override a reading and how that is audited.

Build tasks

1. Add MinIO to compose; scaffold the Reading Service with `reading_db`.
2. Model billing periods per zone (OPEN / CLOSED) with reading windows, and reader-to-route assignments.
3. Implement single and batch submission with an `Idempotency-Key` header (Appendix B).
4. Implement the photo flow: request pre-signed URL -> client uploads -> attach object key to reading.
5. Build the validation engine as independent, pluggable rules. Each rule is a pure function with its own tests.
6. Implement the review queue: list flagged readings, approve, override value (with reason), or reject & request re-read.
7. Generate estimated readings for no-access codes and when the period closes with unread meters.
8. Expose `GET /periods/:id/consumption`: final approved consumption per account, ready for billing.

9. Add Reading Service routes to the gateway and permissions to the matrix tests.

Deliverables

- Reading Service, updated compose and gateway config, rule test suite, ADR on data ownership.

Definition of Done - you are finished when...

- Every rule R1-R7 has table-driven tests covering at least one pass and one fail case.
- Submitting the same batch twice creates **no** duplicate readings, and the second response equals the first.
- The meter-replacement consumption formula is proven by a test.
- A reader cannot submit readings for a route not assigned to them.
- Closing a period is blocked while flagged readings are unresolved (or the override is explicitly recorded).

Common pitfalls

- Letting the Reading Service write to the Customer Service's database "just this once".
- Validation rules tangled into one giant function. Keep them separate and composable.

Stretch goals (optional)

- Customer self-reading submissions (marked with lower trust).
- A route optimisation endpoint that orders stops by address or coordinates.

Reflection questions for your dev journal

? If the Customer Service is down, can readers still submit? Should they? What did you choose?

? What does "idempotent" mean, and why does an offline mobile app make it essential?

Milestone 5 - Billing Engine

MILESTONE 5 | WEEKS 10-12

Goal: Generate accurate, reproducible and auditable bills for an entire billing period from approved consumption and versioned tariffs, and maintain a per-account ledger that is always the source of truth for balances.

Billing is where software bugs cost real money and customer trust. This milestone teaches correctness: exact money arithmetic, immutability, versioning and jobs that are safe to run twice. It is the longest milestone for a reason.

Concepts you will learn

Money in integer minor units / decimals	Rounding rules	Tiered (block) pricing
Effective-dated versioning	Immutable documents	Double-entry style ledgers
Idempotent batch jobs	Background job queues	PDF generation
Property-based testing		

What you are given

- The sample tariff (3.3) and the worked example (3.4), which must match to the cent.
- Issued bills are **immutable**. Corrections are adjustments (credit/debit memos) that create new ledger entries.
- A tariff version that has been used by an issued bill can never be edited, only superseded by a new version.
- Account balance is always computed from ledger entries, never stored as a free-floating number.

What you decide (your responsibility)

- The tariff data model (how tiers, fees and taxes are represented and versioned by effective date and classification).
- Rounding strategy (per line vs. on totals) and money representation. Document it clearly.
- Bill-number format and due-date rules.
- Whether a bill run is a synchronous request or a background job (BullMQ on Redis is a good option). Write an ADR.
- Your PDF approach (pdfkit, React-PDF, or HTML + headless Chromium) and your bill design.

Build tasks

1. Scaffold the Billing Service with `billing_db`: tariffs, tariff_versions, tiers, bill_runs, bills, bill_lines, ledger_entries, adjustments.
2. Implement tariff management with effective dating. Seed Residential, Commercial and Industrial tariffs.
3. Write the charge calculator as a **pure function**: (consumption, tariff version, account context) -> bill lines. No I/O.
4. Implement the bill run: preview (draft bills + summary) -> review -> issue. Pull consumption from the Reading Service for now.
5. Carry arrears forward and apply penalties via a scheduled job for overdue balances.
6. Implement adjustments / credit memos with reason codes and approver.

7. Generate PDF bills (account, period, readings, consumption, line items, balance, due date) and store them in MinIO.
8. Expose ledger and statement endpoints: `GET /accounts/:id/ledger`, `GET /bills/:id/pdf`.
9. Record which reading IDs and which tariff version produced each bill (traceability).

Deliverables

- Billing Service, calculator test suite, sample PDF bill, ADRs on money handling and job strategy.

Definition of Done - you are finished when...

- The worked example in 3.4 and the two extra cases produce exactly $2,037.81, $324.80 and $2,567.04.
- Running the same bill run twice (or retrying after a crash mid-run) never produces duplicate bills.
- Publishing a new tariff version affects only periods on or after its effective date.
- For any bill you can show the readings and tariff version used, and every line recomputes identically.
- A bill run for 1,000 seeded accounts completes in under 60 seconds locally.

Common pitfalls

- `0.1 + 0.2 !== 0.3`. Never use JavaScript floats for money.
- UPDATE-ing an issued bill to fix a mistake.
- Putting tariff numbers in code instead of data.

Stretch goals (optional)

- Prorate charges when a tariff changes in the middle of a period.
- Property-based tests (fast-check): bill total always equals the sum of its lines; more consumption never costs less.
- Bill-run report export (CSV) for finance.

Reflection questions for your dev journal

? Why do accountants never erase a ledger entry? How does your design respect that?

? What would happen if the Reading Service changed a reading after the bill was issued?

Milestone 6 - Next.js Staff Portal

MILESTONE 6 | WEEKS 13-15

Goal: A polished, secure and accessible staff portal in which admins and billing officers manage customers and meters, review flagged readings, run and issue bills and monitor operations on a dashboard.

You now have a working backend but no way for real users to use it. This milestone is also your biggest creative space: the information design, look and feel and workflow ergonomics are yours. Aim for a tool people could use for 8 hours a day.

Concepts you will learn

Next.js App Router	Server vs client components	Backend-for-Frontend (BFF)
httpOnly cookie sessions	Middleware route protection	Data fetching & caching
Forms & server actions	Accessible UI (WCAG)	Design systems

What you are given

- Required pages: login; dashboard; customers & accounts (list/detail with meter history, readings, bills, ledger); reading review queue with photo viewer; tariffs; billing periods & bill runs (preview -> issue); users & roles; audit log.
- Tokens are never readable by browser JavaScript (httpOnly, Secure, SameSite cookies).
- Navigation and actions follow the permission matrix. The UI hides what a role cannot do, and the backend still enforces it.

What you decide (your responsibility)

- UI component library and visual design. Create a small style guide page (colours, type, components).
- Data fetching approach: server components + server actions, TanStack Query, or a mix. Justify it.
- Whether the browser talks to the gateway directly or only through Next.js route handlers (BFF). Write an ADR.
- Dashboard KPIs and charts. Suggested: reading progress per route, flagged count, bills issued, amount billed vs collected.

Build tasks

1. Create `apps/web` with TypeScript, linting and your chosen UI kit; add it to compose.
2. Implement login/logout with httpOnly cookie sessions, silent refresh and a middleware that guards staff routes.
3. Build the app shell: role-aware navigation, breadcrumbs, toasts, error boundaries, loading skeletons.
4. Build the customers/accounts area with server-side search, filters and pagination; account detail with tabs.
5. Build the reading review queue with the photo, the previous 6 readings (a small chart) and approve/override/reject actions.
6. Build tariff management and the bill-run wizard: choose period -> preview totals & exceptions -> issue.
7. Build the dashboard and the users & roles admin screens.
8. Design empty, loading and error states for every page.
9. Write Playwright tests for login, the review queue and a bill run.

Deliverables

- Staff portal in `apps/web`, style guide page, Playwright tests, ADR on BFF vs direct calls.
- A 5-minute demo video of a billing officer's day.

Definition of Done - you are finished when...

- A meter_reader or customer who logs in cannot reach any staff route, even by typing the URL.
- No access token appears in `localStorage`, in client-side JS bundles or in page HTML.
- All large lists paginate on the server; the accounts page stays fast with 10,000 seeded accounts.
- Lighthouse accessibility score of 90 or more on the main pages; the portal is fully usable by keyboard.
- Every page has deliberate empty, loading and error states.

Common pitfalls

- Marking everything `'use client'` and losing the benefits of server components.
- Fetching all records and paginating in the browser.

Stretch goals (optional)

- Dark mode; keyboard shortcuts for the review queue; CSV exports; internationalisation.

Reflection questions for your dev journal

? Which parts of your UI are server components and which are client components? Why?

? What did you learn from watching someone else try your portal for 5 minutes?

Milestone 7 - Event-Driven Architecture: Broker, Payments & Notifications

MILESTONE 7 | WEEKS 16-18

Goal: Services communicate through a message broker using well-defined domain events; the new Payment and Notification services are fully event-driven, and the system stays correct when services crash, restart or receive the same message twice.

This is the milestone where the system becomes a real distributed system. You will learn why simply "publishing a message after saving to the database" loses data, how the outbox pattern fixes it, and why every consumer must be idempotent. These are the ideas that separate microservices that work from microservices that fail.

Concepts you will learn

Publish/subscribe & queues	At-least-once delivery	Idempotent consumers
Transactional outbox	Dead-letter queues & retries	Eventual consistency
Event versioning	Webhooks & signature verification	Choreography vs orchestration (sagas)

What you are given

- The event envelope below and the event catalog in Appendix C.
- Payments are processed at most once per idempotency key; a payment is never credited twice.
- Every consumer handles duplicates and out-of-order delivery safely.
- Locally, email goes to Mailpit (a fake inbox). No real messages are sent during development.

What you decide (your responsibility)

- Broker: RabbitMQ recommended; NATS, Redis Streams or Kafka if you can justify them (ADR).
- Exchange/topic/queue topology and naming.
- Which synchronous calls to replace with events. At minimum, Billing should build its own consumption read model from `reading.*` events instead of calling the Reading Service.
- Payment provider: build a mock gateway service with webhooks, or use a real provider's test/sandbox mode.
- Notification templates, channels and per-customer preferences.

Build tasks

1. Add RabbitMQ and Mailpit to compose. Create `packages/contracts` with typed event schemas (Zod) and a small publish/consume helper.
2. Implement a transactional outbox in Reading and Billing: write the event row in the same DB transaction, and a relay publishes it.
3. Publish reading and billing events; move Billing to an event-fed consumption read model; update your Milestone 4 ADR.
4. Scaffold the Payment Service: over-the-counter payments (staff), online checkout via the gateway/mock, webhook endpoint with signature verification, receipts.
5. Billing consumes `payment.received` and credits the ledger; overpayments become credit balance.

6. Scaffold the Notification Service: consume bill.issued, payment.received, bill.overdue, reading.flagged; render templates; send email; keep a delivery log.
7. Configure retries with exponential backoff and dead-letter queues; build a small CLI or admin page to inspect and replay DLQ messages.
8. Store processed event IDs per consumer (inbox table) to guarantee idempotency.
9. Show payment status in the staff portal, including *pending* states caused by eventual consistency.

Event envelope (given)

```
{
  "id": "0b8f7c7e-...-uuid",              // unique; consumers use it for de-duplication
  "type": "billing.bill.issued",          // <context>.<entity>.<past-tense-verb>
  "version": 1,                           // schema version of `data`
  "occurredAt": "2026-10-05T08:30:00Z",
  "source": "billing-service",
  "correlationId": "req-7f3a...",         // ties the event to the originating request/trace
  "data": { "billId": "...", "accountId": "...", "amountDue": 203781, "dueDate": "2026-10-20" }
}
```

Deliverables

- Payment and Notification services, contracts package, outbox/inbox implementation, DLQ tooling.
- An updated architecture diagram and an event-flow diagram. ADRs on broker choice and event design.

Definition of Done - you are finished when...

- Stop the Billing Service, record 5 payments, start it again: all 5 are credited exactly once.
- Deliver the same `payment.received` event twice by hand: the balance changes once.
- Kill the Reading Service right after a DB commit but before publishing: the event is still delivered after restart (outbox).
- A malformed ("poison") message ends up in the DLQ after N retries and does not block the queue.
- Issuing bills produces emails in Mailpit with the correct amount and a PDF link.

Common pitfalls

- Publishing to the broker inside the DB transaction, or after it without an outbox.
- Putting whole entities in events "just in case". Publish facts, not databases.
- Retrying forever with no backoff and no DLQ.

Stretch goals (optional)

- A saga for refunds (Payment -> Billing adjustment -> Notification) with compensation on failure.
- Schema registry or contract tests for events.
- SMS channel via a provider's sandbox.

Reflection questions for your dev journal

? Where did you accept eventual consistency, and how does the UI explain it to users?

? When would orchestration (a central coordinator) be better than choreography (services reacting to events)?

Milestone 8 - Customer Portal & Field Reader App

MILESTONE 8 | WEEKS 19-20

Goal: Customers can view their usage and bills and pay online through a self-service portal, and meter readers can complete a full route on a phone with no connectivity, syncing safely when they are back online.

Two very different users, two very different UX problems. The reader app teaches offline-first engineering: local queues, background sync and conflict handling. It relies on the idempotent batch API from Milestone 4. The customer portal teaches you to design for non-experts on mobile.

Concepts you will learn

Progressive Web Apps	Service workers & caching strategies	IndexedDB offline queues
Background sync	Camera capture on the web	Mobile-first responsive design
Optimistic UI	Conflict resolution	Public-facing UX & accessibility

What you are given

- Reader must-haves: today's route in walking order, search by meter serial/address, previous reading and a sanity warning while typing, photo capture, no-access codes, offline queue with visible sync status.
- Customer must-haves: register & link account, current bill and history, PDF download, 12-month usage chart, online payment, notification preferences.
- Customers can only ever see their own accounts (verify it in tests).

What you decide (your responsibility)

- One Next.js app with route groups, or separate apps per audience. Write an ADR on the trade-offs (bundle size, deployment, shared auth).
- Offline storage library (idb, Dexie) and caching strategy for route data.
- What happens when an offline reading conflicts with one entered by an officer in the meantime.
- Branding and tone of voice for customer-facing screens.

Build tasks

1. Make the reader area installable (manifest, icons) and add a service worker that caches the app shell and today's route.
2. Store submissions in an IndexedDB outbox; sync in idempotent batches when online; show pending/synced/failed per stop.
3. Implement camera capture with client-side image compression before upload.
4. Build the customer area: onboarding/linking, dashboard (amount due, due date, usage chart), bill history, PDF download.
5. Implement online payment end-to-end through the Payment Service, with clear pending -> confirmed states.
6. Implement notification preferences, stored in the Notification Service.
7. Write Playwright tests with an emulated mobile device and offline mode.

Deliverables

- Reader PWA and customer portal, offline E2E test, ADR on app structure.

Definition of Done - you are finished when...

- In airplane mode, a reader completes 20 stops with photos; after reconnecting, everything syncs with **zero** duplicates.
- A customer pays a bill and sees it marked pending, then paid, without refreshing manually (polling or SSE).
- All customer and reader screens are usable at 360 px width.
- A customer changing an account ID in the URL gets 403/404, never another person's data.

Stretch goals (optional)

- Push notifications for bill reminders (Web Push).
- Leak/incident reporting with location from the customer app.
- Server-Sent Events for live dashboard updates in the staff portal.

Reflection questions for your dev journal

? What did offline-first force you to change in your backend design?

? How did designing for customers differ from designing for staff?

Milestone 9 - Quality Gates & CI/CD

MILESTONE 9 | WEEKS 21-22

Goal: Every pull request is automatically linted, type-checked, tested and security-scanned, and every merge to `main` produces versioned container images in a registry. `main` is always deployable.

With seven services, manual testing no longer scales. You will build the safety net that lets you change things quickly with confidence. You will also learn how contract tests catch integration breaks between services without running the whole system.

Concepts you will learn

Test pyramid	Consumer-driven contract tests (Pact)	End-to-end tests
GitHub Actions workflows	Build caching & affected-only builds	Container registries & image tagging
Supply-chain scanning	Branch protection	Semantic versioning & changelogs

What you are given

- PR checks must include lint, type-check, unit and integration tests for affected services.
- Images are tagged with the git SHA (and a semver tag on release). The tag `latest` is never used for deployments.
- No secrets are committed to workflows. Use repository/environment secrets.

What you decide (your responsibility)

- Branching strategy (trunk-based recommended) and release/versioning scheme.
- Which tests run on every PR, on merge, and nightly.
- How to detect affected services in the monorepo (Turborepo, Nx, or path filters).

Build tasks

1. Create a PR workflow: install with cache, lint, type-check, unit tests, integration tests using service containers or Testcontainers.
2. Add consumer-driven contract tests for at least two relationships (e.g., web -> gateway -> billing, billing -> reading events).
3. Create an E2E workflow that starts the stack with compose and runs the *life of a bill* Playwright test (Section 2.4).
4. Build and push images for changed services to GHCR on merge, tagged by SHA.
5. Add scanning: dependency audit, container image scan (Trivy/Grype), secret scanning; fail on critical issues.
6. Enable branch protection with required checks; add Dependabot or Renovate.
7. Automate changelogs/releases from Conventional Commits.

Deliverables

- `.github/workflows/*`, contract tests, E2E test, CI badge in the README, ADR on test strategy.

Definition of Done - you are finished when...

- A PR with a failing test or lint error cannot be merged.
- A PR touching only the Notification Service does not rebuild or retest the other services.
- Median PR pipeline time is under 10 minutes.
- Breaking a contract (e.g., renaming a field Billing relies on) fails CI before merge.
- Every image in the registry can be traced to its commit.

Stretch goals (optional)

- Preview environments per PR (compose on a runner, or a K8s namespace after Milestone 10).
- Mutation testing (Stryker) on the billing calculator.
- Signed images (cosign) and an SBOM.

Reflection questions for your dev journal

? Which bug would each layer of your test pyramid have caught? Which one would none of them catch?

Milestone 10 - Kubernetes & Infrastructure as Code

MILESTONE 10 | WEEKS 23-25

Goal: Run the complete AquaFlow system on a Kubernetes cluster that is described entirely in code and deployed automatically from Git, locally first and optionally in a cloud environment provisioned with Terraform.

Docker Compose is for one machine. Kubernetes is how most organisations run microservices in production. You will learn how Kubernetes keeps services healthy, rolls out new versions safely and scales them, and how Infrastructure as Code (IaC) makes environments reproducible.

Concepts you will learn

Pods, Deployments, Services	Ingress	ConfigMaps & Secrets
Liveness/readiness probes	Requests, limits & autoscaling (HPA)	Namespaces & environments
Helm / Kustomize	GitOps (Argo CD)	Terraform state & modules

What you are given

- Everything needed to run the system is declared in the repository. No manual `kubectl edit`.
- No plaintext secrets in Git (use Sealed Secrets, SOPS or External Secrets).
- Every service has readiness and liveness probes and resource requests/limits.
- Database migrations run as Kubernetes Jobs (or init containers) before new app versions receive traffic.

What you decide (your responsibility)

- Helm charts vs Kustomize overlays (ADR). A single reusable "service" chart is a good pattern.
- In-cluster Postgres/RabbitMQ (via charts or operators) vs managed cloud services. Write down the trade-offs.
- Ingress controller (ingress-nginx, Traefik) and local DNS approach.
- Whether to go to the cloud, and which provider. **Optional**, see the cost warning below.

Build tasks

1. Script local cluster creation with kind or k3d, including an ingress controller and local registry.
2. Write manifests/charts for every service and the web app; deploy Postgres, RabbitMQ, Redis and MinIO via charts.
3. Configure ingress: `app.aquaflow.local` -> web, `api.aquaflow.local` -> gateway.
4. Implement secrets management and environment overlays (dev, staging).
5. Add HPA to the Reading Service and the gateway; add PodDisruptionBudgets.
6. Install Argo CD and point it at your repo (GitOps), or deploy from GitHub Actions. CI updates image tags; the cluster syncs.
7. Practise operations: rolling update, rollback, scaling, draining a node, reading pod logs and events.
8. Optional cloud: Terraform for network, a small managed K8s cluster (or a single VM with k3s) and managed Postgres; remote state; **destroy when done**.

Deliverables

- `infra/k8s`, cluster bootstrap script, GitOps config, optional `infra/terraform`.
- A runbook: *how to deploy, roll back and debug a failing pod*.

Definition of Done - you are finished when...

- `make cluster-up` goes from nothing to a fully working system with seeded data.
- A rolling update of the Billing Service during a k6 smoke test causes zero failed requests.
- Deleting any pod results in automatic recovery; the system keeps serving.
- Merging a change to `main` reaches the cluster with no manual steps.
- `git grep` finds no plaintext passwords or keys.

Common pitfalls

- Running the database as a plain Deployment with no persistent volume.
- Readiness probes that only check "process is up" and ignore dependencies (and liveness probes that check dependencies, which cause restart storms).
- Leaving cloud resources running. Set a budget alert on day one.

Stretch goals (optional)

- Network policies so only the gateway can reach services, and only services can reach their DB.
- A service mesh (Linkerd) for mTLS between services.
- Blue/green or canary deployments with Argo Rollouts.

Reflection questions for your dev journal

? What does Kubernetes do for you that Docker Compose did not? What new complexity did it add?

? If a teammate deleted the whole cluster, how long would it take you to rebuild it?

Milestone 11 - Observability & Resilience

MILESTONE 11 | WEEKS 26-27

Goal: When something goes wrong you can find what is broken and why within minutes using traces, metrics and logs, and the system degrades gracefully instead of failing all at once.

In a distributed system a single user action touches many services and a broker. Without observability, debugging is guesswork. You will instrument everything, define what "healthy" means as SLOs, and then break the system on purpose to prove it recovers.

Concepts you will learn

Logs, metrics, traces	OpenTelemetry	Context propagation through messages
RED & USE methods	Business metrics	SLIs, SLOs & error budgets
Alerting	Timeouts, retries, circuit breakers	Chaos engineering / game days

What you are given

- Every service emits OpenTelemetry traces and metrics, and structured logs that include the trace ID.
- Trace context must pass through RabbitMQ messages, so one trace spans sync and async hops.
- At least three SLOs are defined, measured and alerting.

What you decide (your responsibility)

- Your SLOs. Suggestions: 99% of reading submissions under 500 ms; a bill run for 10k accounts under 10 minutes; 99.9% of payments credited within 60 s.
- Dashboard design: what an on-call engineer needs at 3 a.m. versus what a manager wants to see.
- Which calls get circuit breakers, retry policies and fallbacks.

Build tasks

1. Add OpenTelemetry SDK + auto-instrumentation (HTTP, pg, amqplib, Redis) to all services and the Next.js server.
2. Deploy an observability stack: OTel Collector, Prometheus, Grafana, Loki, Tempo or Jaeger (compose and K8s).
3. Propagate trace context in event headers; link logs to traces in Grafana.
4. Build dashboards: per-service RED metrics, broker queue depth/DLQ size, and business KPIs (readings per hour, bills issued, amount collected).
5. Define SLOs and alert rules; route alerts to a local webhook, email or chat.
6. Add timeouts everywhere, retries with jitter where safe, and a circuit breaker (e.g., opossum) on remaining sync calls.
7. Run a game day: kill services, add network latency, fill the DLQ. Write a short post-incident report for each experiment.

Deliverables

- Instrumented services, dashboards as code (JSON in Git), alert rules, SLO document, game-day report.

Definition of Done - you are finished when...

- You can open **one trace** that follows a customer's online payment from the browser click through gateway, Payment, broker, Billing and Notification.
- From a log line you can jump to its trace, and back.
- The business dashboard shows today's readings, bills and collections.
- With the Customer Service down, reading submission still works or fails fast with a clear message, as your ADR says.
- The game-day report lists at least 3 experiments, what you expected, what happened and what you fixed.

Stretch goals (optional)

- Error budget burn-rate alerts.
- Continuous profiling (Pyroscope).
- Synthetic monitoring of the customer portal.

Reflection questions for your dev journal

? Which of your alerts would actually wake someone up, and should it?

? What was the most surprising thing your game day revealed?

Milestone 12 - Hardening, Performance & Capstone

MILESTONE 12 | WEEKS 28-29 (2.5 WEEKS)

Goal: Prove that AquaFlow is secure, performs at the target load, can be recovered from disaster and is documented well enough for another developer to run and extend. Then present it.

Professional work is not finished when the features work. It is finished when others can trust it, operate it and build on it. This milestone ties everything together and ends with a capstone presentation of your journey.

Concepts you will learn

OWASP Top 10	Authorization testing (IDOR)	Security headers & CSP
Load testing (k6)	Query optimisation & indexing	Backup & restore
Technical writing	Presenting architecture	

What you are given

- The Capstone Acceptance Criteria in Section 6 are the final bar.
- Load target: 10,000 accounts, 50 concurrent readers syncing, 200 concurrent customer portal users.

What you decide (your responsibility)

- How you present your work: live demo, recorded video, slides, or a written case study.
- Which improvements to prioritise in the time you have. Make the trade-off explicit.

Build tasks

1. Run an OWASP Top 10 review; write an automated authorization test suite that tries to access other users' data on every endpoint.
2. Add security headers and a Content Security Policy in Next.js and the gateway; enforce request size limits.
3. Rotate all secrets and the JWT signing key using your runbooks.
4. Write k6 load tests for reading sync, bill run and portal browsing; find and fix the top 3 bottlenecks (indexes, N+1 queries, caching).
5. Automate Postgres backups and **perform a restore drill** into a fresh cluster; record how long it took.
6. Finish documentation: README, architecture overview, ADR index, API docs, runbooks, onboarding guide.
7. Write a retrospective: what went well, what you would change, what you would build next.
8. Present the capstone to your mentor or team.

Deliverables

- Security review document, load test scripts & results, backup/restore runbook, complete docs, retrospective, capstone presentation.

Definition of Done - you are finished when...

- Every item in the Capstone Acceptance Criteria (Section 6) passes.
- A developer who has never seen the project can run it locally using only the README, in under 30 minutes.
- Your presentation explains at least three decisions you would make differently now, and why.

6. Capstone Acceptance Criteria

This is the definition of **done** for the whole project. Your mentor will go through this list with you at the final checkpoint.

6.1 Functional

- The *life of a bill* (Section 2.4) works end to end on Kubernetes: reading (offline) -> validation -> review -> bill run -> PDF -> email -> online payment -> ledger updated.
- The worked billing example and edge cases (Section 3.4) are part of the automated test suite and pass.
- All four roles behave according to the permission matrix; automated authorization tests prove it.
- Meter replacement, estimated readings, arrears, penalties and adjustments are all demonstrable.

6.2 Architecture & code

- Six backend services plus the gateway, each with its own database, independently buildable and deployable.
- No service reads another service's database. Integration is via APIs and events only.
- Outbox and idempotent consumers are in place; duplicate or replayed events cause no double effects.
- 12+ ADRs documenting the significant decisions, including at least one decision you later reversed.

6.3 Infrastructure & operations

- One command starts the full stack locally (Compose) and one command creates the local Kubernetes environment.
- CI blocks broken changes; CD delivers merged changes to the cluster automatically.
- No plaintext secrets in Git. Images are scanned, and they run as non-root with resource limits.
- A single distributed trace follows a payment across all services; SLO dashboards and alerts exist.
- A backup has been restored successfully, and the procedure is written as a runbook.

6.4 Assessment rubric

Dimension	Developing	Proficient	Excellent
Functionality	Core flow works with manual help	All given requirements met	Requirements met plus thoughtful stretch goals
Code quality	Works but inconsistent structure	Consistent layering, typed, readable	Clean domain modelling others could extend easily
Architecture decisions	Choices made but not explained	ADRs explain options and trade-offs	ADRs revisited as understanding grew; honest about costs
Testing	Some unit tests	Unit, integration, E2E in CI	Contract, property-based and load tests; tests used to drive design
Infrastructure & ops	Runs in Compose	Runs on K8s via GitOps, observable	Resilience proven by game days; restore drill done

Dimension	Developing	Proficient	Excellent
Communication	Sparse journal and docs	Clear README, runbooks, regular demos	Docs a new developer can onboard from alone; strong presentation

6.5 Mentor checkpoints

Checkpoint	When	Focus of the review
A	End of M0	Domain understanding, context map, service boundaries
B	End of M2	Code structure, testing habits, security basics
C	End of M5	Billing correctness, data modelling, ADR quality
D	End of M7	Distributed-systems reasoning: failure modes, idempotency, consistency
E	End of M10	Infrastructure design, deployment workflow, secrets
F	End of M12	Capstone presentation and rubric assessment

If you fall behind

- Stretch goals are optional. Skip them first.
- Milestone 8 can be reduced to the reader PWA only; the customer portal can become a single bills page.
- Milestone 10's cloud deployment is optional; local Kubernetes is enough to meet the capstone criteria.
- Do **not** skip tests, ADRs or the journal. They are the learning.

Appendix A - Glossary of Technical Terms

Term	Meaning
ADR	Architecture Decision Record: a short document capturing a decision, its context, options and consequences.
Bounded context	A boundary within which a domain model and its terms have one consistent meaning; often maps to a service.
BFF	Backend-for-Frontend: a server layer tailored to one UI (here, Next.js route handlers) that calls backend services.
Circuit breaker	Stops calling a failing dependency for a while so failures do not cascade.
Dead-letter queue	Where messages go after repeated processing failures, for inspection and replay.
Eventual consistency	Data across services becomes consistent after a short delay rather than instantly.
GitOps	The desired state of the cluster lives in Git; an agent (e.g., Argo CD) continuously applies it.
Idempotency	Performing an operation multiple times has the same effect as performing it once.
JWKS	JSON Web Key Set: a public endpoint listing the keys used to verify JWT signatures.
Outbox pattern	Writing events to a DB table in the same transaction as the business change, then publishing them reliably.
SLI / SLO	Service Level Indicator (a measurement) / Objective (the target for that measurement).
Trace / span	A trace is the full journey of one request; spans are the individual timed operations within it.

Appendix B - API Conventions

Paths & naming

- Plural nouns, kebab-case: `/api/v1/billing-periods/:id/bill-runs`. JSON fields in camelCase.
- IDs are UUIDs (v7 recommended for sortability); human-friendly numbers (account no., bill no.) are separate fields.
- Money is sent as integer minor units plus a currency code: `{ "amount": 203781, "currency": "USD" }`.
- Timestamps in ISO-8601 UTC; dates without time (due dates) as `YYYY-MM-DD`.

Error format

```
HTTP/1.1 409 Conflict
{
  "error": {
    "code": "METER_ALREADY_INSTALLED",
    "message": "Account RES-001-0042 already has an active meter (SN 8812345).",
    "details": [{ "field": "meterSerial", "issue": "must be replaced, not added" }],
    "requestId": "req-7f3a2c..."
  }
}
```

Pagination, filtering & idempotency

```
GET /api/v1/accounts?zone=Z1&status=ACTIVE&limit=50&cursor=eyJpZCI6...
-> { "data": [...], "page": { "nextCursor": "eyJpZCI6...", "limit": 50 } }

POST /api/v1/readings/batch
Idempotency-Key: 5c1f0c1e-8a2d-4f7e-9b0e-2d6a4c1d9f11
-> Replaying the same key + same body returns the original response (200/201), never a duplicate.
-> Same key + different body returns 422 IDEMPOTENCY_KEY_REUSED.
```

Status codes you should use

Code	When
200 / 201 / 204	Success / created (with Location header) / success with no body
400	Malformed or invalid input (field-level details included)
401 / 403	Not authenticated / authenticated but not allowed
404	Resource not found, or hidden from this user
409	Conflicts with current state (duplicate, invalid state transition)
422	Semantically invalid (e.g., reading date outside the period window)
429	Rate limit exceeded (include Retry-After)
503	Dependency unavailable (from the gateway or a service with an open circuit)

Appendix C - Event Catalog (starter)

Extend this catalog as you design. Event names are past-tense facts. Payloads carry IDs and the facts consumers need, not whole database rows.

Event type	Producer	Typical consumers	Key data
customer.account.opened	Customer	Reading, Billing	accountId, accountNo, classification, zoneId
customer.account.status-changed	Customer	Reading, Billing, Notification	accountId, from, to, reason
customer.meter.installed	Customer	Reading	accountId, meterId, serial, initialReading, date
customer.meter.replaced	Customer	Reading, Billing	accountId, oldMeterId, finalReading, newMeterId, initialReading
reading.reading.flagged	Reading	Notification (staff)	readingId, accountId, flags[]
reading.reading.approved	Reading	Billing	readingId, accountId, periodId, value, consumption, estimated
reading.period.closed	Reading	Billing	periodId, zoneId, accountCount
billing.bill.issued	Billing	Notification, Payment	billId, billNo, accountId, amountDue, dueDate, pdfKey
billing.bill.overdue	Billing	Notification	billId, accountId, overdueAmount, penalty
billing.adjustment.posted	Billing	Notification	adjustmentId, accountId, amount, reason
payment.payment.received	Payment	Billing, Notification	paymentId, accountId, billId?, amount, method, receivedAt
payment.payment.failed	Payment	Notification	paymentId, accountId, reason
auth.user.registered	Auth	Customer, Notification	userId, role, accountNo?

Appendix D - ADR Template

```
# ADR-0007: Billing builds its own consumption read model from reading events

Status:   Accepted  (Proposed | Accepted | Superseded by ADR-00xx)
Date:     2026-11-02
Deciders: <you>, reviewed by <mentor>

## Context
What is the problem? What forces are at play (requirements, constraints, team skills, deadlines)?

## Options considered
1. Synchronous call to Reading Service during bill run
2. Billing consumes reading.reading.approved events into a local table
3. Shared database view (rejected early: violates data ownership)

## Decision
We choose option 2 because ...

## Consequences
+ Bill runs no longer depend on Reading Service availability
+ ...
- Data is eventually consistent; bill run must wait for period.closed
- Extra table and consumer to maintain; must handle replays idempotently
```

Appendix E - Learning Resources

Topic	Resource
Microservices	Sam Newman, *Building Microservices* (2nd ed.); *Monolith to Microservices*
Data & distributed systems	Martin Kleppmann, *Designing Data-Intensive Applications*
Messaging patterns	Hohpe & Woolf, *Enterprise Integration Patterns*; microservices.io pattern pages (outbox, saga)
Domain modelling	Vlad Khononov, *Learning Domain-Driven Design*
Node.js practice	The *Node.js Best Practices* GitHub repository; official Node.js and Fastify docs
Frontend	Official Next.js docs (App Router, caching, authentication guides); web.dev on PWAs & accessibility
Operations	*The Twelve-Factor App*; Google *Site Reliability Engineering* book (SLO chapters, free online)
Kubernetes & IaC	Kubernetes official tutorials; Helm and Argo CD docs; Terraform tutorials (HashiCorp Developer)
Observability	OpenTelemetry JavaScript docs; Grafana LGTM stack tutorials
Security	OWASP Top 10 and OWASP ASVS; OWASP cheat sheets (JWT, authentication, session management)

A final word

You will get stuck, rewrite things and change your mind. Those are not signs of failure; they are the journey. By the end you will have built a system with the same moving parts as real production platforms, and, more importantly, you will be able to explain *why* every part exists. Good luck, and enjoy it.